

Bank Management System

Python OOP Concepts — Practice Task / Assignment

Is task ka goal hai ek chhota-sa **Bank Management System** Python mein banate hue OOP (Object-Oriented Programming) ke saare major concepts practically cover karna — Class & Object, Constructor, Encapsulation, Inheritance, Polymorphism, Abstraction, Class/Static methods, Magic (Dunder) methods, aur Exception Handling.

Language	Python 3.x
Difficulty	Beginner → Intermediate
Estimated Time	3 – 5 hours
Topics Covered	Full OOP: Class, Object, Encapsulation, Inheritance, Polymorphism, Abstraction, Magic Methods, Exceptions

Note: Ye ek self-practice assignment hai. Har section mein requirement diya gaya hai — code khud likhna hai. Hints aur sample structure diye gaye hain taaki concept clear rahe.

Project Overview

Tumhe ek **Bank Management System** banana hai jisme multiple account types (Savings, Current), customers, transactions (deposit/withdraw/transfer), aur bank-level operations honge. Poora system OOP principles follow karte hue design karna hai — koi bhi cheez procedural (sirf functions se) nahi likhni.

High-Level Class Structure

Neeche diya gaya structure ek suggestion hai — tum naming apni marzi se rakh sakte ho, lekin relationships (inheritance/composition) same concept follow karni chahiye:

- **Person** (base class) → **Customer** (inherits Person)
- **Account** (abstract base class) → **SavingsAccount**, **CurrentAccount** (inherit Account)
- **Bank** — customers aur accounts ko manage karta hai (composition/aggregation)
- **Transaction** — har deposit/withdraw/transfer ka record rakhta hai
- Custom Exceptions: **InsufficientFundsError**, **InvalidAmountError**, **AccountNotFoundError**

1. Class & Object Basics

Requirement: Ek Customer class banao jisme customer ka *name*, *age*, *address*, aur *customer_id* attributes ho. Kam se kam 3 customer objects create karke unki details print karo.

Concept Checklist:

- Class definition syntax (class ClassName:)
- Object/Instance creation
- Instance attributes vs Class attributes (e.g. bank ka naam sab customers ke liye common ho — class attribute banao)
- self keyword ka sahi use

2. Constructor (__init__)

Requirement: Account class mein __init__ constructor likho jo *account_number*, *account_holder*, aur *balance* (default = 0) initialize kare. Constructor mein hi validation daalo ki initial balance negative na ho.

3. Encapsulation

Requirement: balance attribute ko **private** banao (double underscore prefix, e.g. __balance) taaki koi direct account.__balance = 999999 karke balance tamper na kar sake. Balance access/modify karne ke liye **getter aur setter methods** ya @property decorator use karo.

Concept Checklist:

- Private attributes (__balance) aur protected attributes (_pin)
- get_balance() method ya @property decorator
- Setter mein validation (e.g. deposit amount > 0 hona chahiye)

Hint — Property Decorator:

```
@property
def balance(self):
    return self.__balance

@balance.setter
def balance(self, amount):
    if amount < 0:
        raise InvalidAmountError("Balance negative nahi ho sakta")
    self.__balance = amount
```

4. Inheritance

Requirement: Account ko base/parent class banao. Usse do child classes inherit karo:

- **SavingsAccount** — extra attribute *interest_rate*, method *add_interest()*
- **CurrentAccount** — extra attribute *overdraft_limit*, jisse balance negative bhi ja sake (limit tak)

Dono child classes mein *super().__init__()* use karke parent constructor call karo. Ek **multilevel inheritance** bhi try karo — e.g. *PremiumSavingsAccount* jo *SavingsAccount* se inherit ho aur extra benefits (locker facility, higher interest) de.

Concept Checklist:

- Single inheritance (Account → SavingsAccount)
- Multilevel inheritance (Account → SavingsAccount → PremiumSavingsAccount)
- *super()* function ka use
- Method overriding (child class mein parent ka method redefine karna)

5. Polymorphism

Requirement: Har account type mein *withdraw()* method ko **override** karo taaki logic alag ho:

- **SavingsAccount.withdraw()** — balance se zyada withdraw allow na ho
- **CurrentAccount.withdraw()** — overdraft limit tak allow ho

Fir ek list mein alag-alag account objects daal kar loop chalo aur sab pe same *account.withdraw(amount)* call karo — dekho kaise har object apna alag behavior dikhata hai (**runtime polymorphism**). Bank class mein bhi ek *calculate_interest(account)* method banao jo different account types ke liye alag result de.

6. Abstraction

Requirement: Python ke *abc* module se Account ko **abstract base class** banao, taaki koi directly Account ka object create na kar sake — sirf iske child classes (*SavingsAccount*, *CurrentAccount*) ke object bane.

```
from abc import ABC, abstractmethod

class Account(ABC):
    @abstractmethod
    def withdraw(self, amount):
        pass

    @abstractmethod
    def account_type(self):
        pass
```

Har child class mein in dono abstract methods ko implement karna compulsory hoga — agar nahi karoge to Python error dega. Isse samajhna hai ki abstraction kaise ek **common interface/contract** force karta hai.

7. Class Methods & Static Methods

Requirement:

- Ek **class attribute** banao `total_accounts` jo track kare ki total kitne accounts bank mein bane hain. Har naya account bante hi ye counter badhna chahiye.
- Ek **@classmethod** banao — `get_total_accounts(cls)` — jo total accounts return kare.
- Ek **@staticmethod** banao — `validate_account_number(acc_no)` — jo check kare ki account number sahi format (e.g. 10 digit numeric) mein hai ya nahi. Ye method kisi object ya class state pe depend nahi karta, isliye static hona chahiye.

Difference samajhna zaroori hai: instance method (self leta hai), classmethod (cls leta hai, class-level data pe kaam karta hai), aur staticmethod (na self na cls — bas ek utility function jo class ke andar rakha gaya hai).

8. Magic / Dunder Methods

Requirement: Apni Account class mein ye special methods implement karo:

- `__str__(self)` — jab `print(account)` karo to readable output aaye (e.g. "SavingsAccount [1023456789] - Balance: 5000")
- `__repr__(self)` — developer-friendly representation
- `__eq__(self, other)` — do accounts ko compare karo (same account number = equal)
- `__add__(self, other)` — do accounts ka balance add karke total return kare (sirf demonstration ke liye, real banking mein aisa nahi hota!)
- `__len__(self)` (Bank class mein) — `len(bank)` call karne pe total customers return ho

9. Custom Exceptions & Exception Handling

Requirement: Built-in Exception class se inherit karke ye custom exceptions banao:

- **InsufficientFundsError** — jab withdraw amount balance (ya overdraft limit) se zyada ho
- **InvalidAmountError** — jab deposit/withdraw amount negative ya zero ho
- **AccountNotFoundError** — jab Bank class mein koi invalid account number search karo

Fir deposit(), withdraw(), aur transfer() methods mein try / except / else / finally block use karke in exceptions ko properly raise aur handle karo. Main program mein bhi user input lete waqt exceptions ko gracefully catch karo (crash nahi hona chahiye).

Hint — Custom Exception:

```
class InsufficientFundsError(Exception):
    def __init__(self, message="Balance is not sufficient"):
        super().__init__(message)
```

10. Bonus Challenges (Optional)

- **Composition:** Bank class ke andar ek list ho self.accounts = [] — ye "has-a" relationship demonstrate karega (Bank *has* Accounts, isse alag hai inheritance se).
- **Transaction History:** Har account apni transactions list maintain kare, aur print_statement() method se poori history dikhaye.
- **File Handling + OOP:** Bank data ko JSON/CSV file mein save aur load karo (persistence).
- **Interface-like Design:** Ek Transferable mixin class banao jisme transfer_to(other_account, amount) method ho, aur usse SavingsAccount/CurrentAccount mein multiple inheritance se add karo.
- **Operator overloading:** __lt__ aur __gt__ implement karo taaki accounts ko balance ke basis pe sort kar sako (sorted(list_of_accounts)).

11. Final Integration — Menu-Driven Program

Sab kuch banane ke baad, ek simple **console menu** banao jisse user interact kar sake:

```
===== BANK MANAGEMENT SYSTEM =====
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Check Balance
6. Print Statement
7. View Total Accounts (classmethod)
8. Exit
Enter your choice:
```

Ye menu ek while True loop mein chale, user ka input lo, aur corresponding method call karo. Har operation try/except mein wrapped ho taaki galat input pe program crash na ho.

Self-Evaluation Checklist

OOP Concept	Kya cover kiya?
Class & Object	■
Constructor (__init__)	■
Encapsulation (private attrs + getter/setter)	■
Inheritance (single + multilevel)	■
Polymorphism (method overriding)	■
Abstraction (ABC + abstractmethod)	■
Class methods & Static methods	■
Magic methods (__str__, __eq__, __add__, etc.)	■
Custom Exceptions + try/except handling	■
Composition (Bank has-a Accounts)	■

Tip: Jab tumhara code sab checkboxes cover kare, matlab tumne Python OOP ka core practically samajh liya hai. Isi structure ko aage badhakar Loan Module, Admin Panel, ya GUI (Tkinter) bhi add kar sakte ho for extra practice.